cQuizy

Tesztdokumentáció

Vizsgaremeket készítették:

Kele Márton Róbert

Molnár Károly

Tartalom

Tartalom....2
1. Frontend Tesztek....3
    1.1. Regisztráció és bejelentkezés....3
    2.2. Csoport és projekt létrehozás....4
2. Backend Tesztek....7
    2.1. Felhasználók....7
    2.2. Csoportok....11
    2.3. Projektek....14
    2.4. Kvízek....18

1. Frontend Tesztek

A Flutter teszteléséhez a beépített tesztelőt használtam amihez a teszteket szintén dart programozási nyelvben kell írni.

1.1. Regisztráció és bejelentkezés

Leírás	Autentikációs folyamatot (Regisztráció, Bejelentkezés) az API-n keresztül.
Előfeltétel	API szerver fut és elérhető
Várt eredmény	Sikeres bejelentkezés és token lekérése
Eredmény	Sikeres

Register and Login flow with test_0 Authentication Flow Integration Test < test\auth_test.dart < cquizy

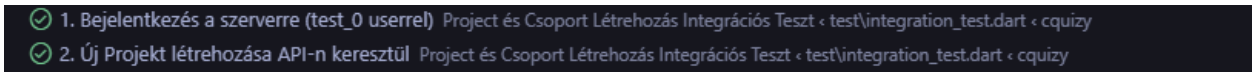
Golden Test

Leírás	Login képernyő vizuális megjelenését
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres

LoginPage Golden Test - Initial State test\golden_login_test.dart < cquizy

2.2. Csoport és projekt létrehozás

Leírás	1. Bejelentkezik egy létező felhasználóval a rendszerbe, amivel szerez egy hozzáférési tokent. 2. Létrehoz egy új Projektet az API-n keresztül, és ellenőrzi, hogy sikeres volt-e.
Előfeltétel	AZ API szerver fut és elérhető a konfigurált címen, illetve létezik egy "test_0" nevű tesztfelhasználó.
Várt eredmény	Minden végpont visszajelzése 200-as vagy 201-es HTTP kód (vagy logikai siker), és a létrehozott entitások adatai megegyeznek a beküldöttel.
Eredmény	Sikeres



Leírás	3. Létrehoz egy új Csoportot az API-n keresztül, és validálja annak sikerességét (pl. pontos színkód ellenőrzése).
Előfeltétel	AZ API szerver fut és elérhető a konfigurált címen, illetve létezik egy "test_0" nevű tesztfelhasználó.
Várt eredmény	Minden végpont visszajelzése 200-as vagy 201-es HTTP kód (vagy logikai siker), és a létrehozott entitások adatai megegyeznek a beküldöttel.
Eredmény	Sikeres

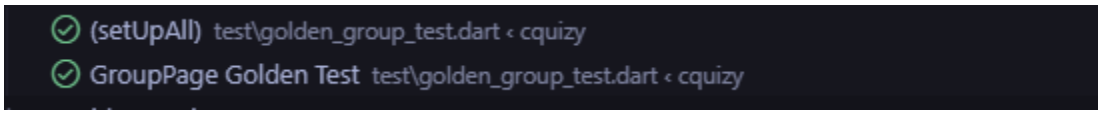
3. Új Csoport létrehozása API-n keresztül Project és Csoport Létrehozás Integrációs Teszt ‹ test\integration_test.dart ‹ cquizy

Felületi vizuális tesztek:

Leírás	A ProjectEditorPage megjelenését teszteli
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres

Dart
(setUpAll) test\golden_project_test.dart ‹ cquizy
ProjectEditorPage Golden Test test\golden_project_test.dart ‹ cquizy

Leírás	A GroupPage vizuális megjelenését teszteli
Előfeltétel	nincs előfeltétel
Várt eredmény	A képernyő megjelenése megegyezik a referenciaképpel.
Eredmény	Sikeres



2. Backend Tesztek

A backend tesztek pytest segítségével lettek elkészítve.

2.1. Felhasználók

Felhasználó regisztrálása

Végpont	`POST /api/users/register`
Leírás	Új felhasználó regisztrálása érvényes adatokkal.
Előfeltétel	A megadott felhasználónév és email nem szerepel az adatbázisban.
Teszt	

```python
def test_register_user_success(client):
    """Test successful user registration."""
    payload = {
        "username": "new_user",
        "email": "new@example.com",
        "password": "securepassword",
        "first_name": "john",
        "last_name": "doe",
        "nickname": "Johnny"
    }

    response = client.post("/register", json=payload)

    assert response.status_code == 200
    assert "success" in response.json()

    #? Felhasználó létrejött-e
    user = User.objects.get(username="new_user")
    assert user.email == "new@example.com"

    #? Név helyesen lett-e formázva
    assert user.first_name == "John"
```

Várt Eredmény	HTTP 200-as státuszkód. Az új felhasználó bekerül az adatbázisba és a neve automatikusan megformázódik.
Eredmény	Sikeres

Bejelentkezés

Végpont	`POST /api/users/login`
Leírás	Létező felhasználó bejelentkeztetése és hitelesítése JWT-tokennel.
Előfeltétel	A felhasználó létezik, érvényes adatokkal lép be.

Teszt	`def test_login_success(client, test_user):`
	`    """Test login with correct credentials returns a token."""`
	`    payload = {`
	`        "username": "testuser_123",`
	`        "password": "strongpassword123"`
	`    }`
	
	`    response = client.post("/login", json=payload)`
	
	`    assert response.status_code == 200`
	
	`    #? Token létrejött-e`
	`    assert "token" in response.json()`

Várt Eredmény	HTTP 200-as státuszkód. A JWT-tokent visszakapja a felhasználó.
Eredmény	Sikeres

Felhasználó adatainak védelme

Végpont	`POST /api/users/me`
Leírás	Annak ellenőrzése, hogy az API blokkolja a token nélküli kéréseket a védett végpontokon.
Előfeltétel	A lekéréshez nincs csatolva a JWT-token.

Teszt	`def test_get_me_unauthenticated(client):`
	`  """Test getting current user data fails without a token."""`
	`  response = client.get("/me")`
	
	`  assert response.status_code == 401`

Várt Eredmény	HTTP 401-as státuszkód. A lekérést a backend elutasítja.
Eredmény	Sikeres

Végpont	`POST /api/users/me`
Leírás	Annak ellenőrzése, hogy a bejelentkezett felhasználó saját adatait sikeresen le tudja kérni.
Előfeltétel	A lekéréshez csatolva lett a JWT-token.

Teszt	`def test_get_me_authenticated(client, auth_headers, test_user):`
	`  """Test getting current user data with a valid JWT token."""`
	`  response = client.get("/me", headers=auth_headers)`
	
	`  assert response.status_code == 200`
	`  data = response.json()`
	`  assert data["username"] == test_user.username`
	`  assert data["email"] == test_user.email`

Várt Eredmény	HTTP 200-as státuszkód. A lekérést a backend így már elfogadja.
Eredmény	Sikeres

Felhasználó törlése

Végpont	`DELETE /api/users/me`
Leírás	A fióktörlés ellenőrzése a jelszó megadása után, illetve a "soft-delete" végrehajtásának ellenőrzése.
Előfeltétel	A felhasználó be van jelentkezve, van érvényes tokenje és a kérés törzsében a helyes jelszót küldi el.

Teszt	`def test_delete_account(client, auth_headers, test_user):`
	`    """Test soft-deletion of the user account."""`
	`    payload = {`
	`        "password": "strongpassword123"`
	`    }`
	
	`    response = client.delete("/me", json=payload, headers=auth_headers)`
	
	`    assert response.status_code == 200`
	
	`    # Felhasználó újratöltése, létezik e még`
	`    test_user.refresh_from_db()`
	
	`    assert test_user.is_active is False`
	`    assert test_user.date_deleted is not None`
	`    #? E-mail anonymizálva lett-e`
	`    assert test_user.email != "test@email.com"`

Várt Eredmény	Az adatbázisban a rekord nem törlődik fizikailag, hanem az `is_active` mező `False`-ra vált, a `date_deleted` kitöltésre kerül a jelenlegi időbélyeggel, az e-mail cím és a nevek pedig anonimizálásra kerülnek.
Eredmény	Sikeres

2.2. Csoportok

Csoport létrehozása

Végpont	`POST /api/groups`
Leírás	Új csoport létrehozása. A teszt ellenőrzi a végpont működését és a jogosultságok automatikus kiosztását.
Előfeltétel	A felhasználó be van jelentkezve (érvényes JWT token).
Teszt	

```
def test_create_group(groups_client, auth_headers_groups,
test_user_groups):
    """Test creating a new group."""
    payload = {
        "name": "Matek 6/B",
        "color": "#FF5733"
    }

    response = groups_client.post("/", json=payload,
headers=auth_headers_groups)

    assert response.status_code == 200
    data = response.json()
    assert data["name"] == "Matek 6/B"
    assert data["color"] == "#FF5733"
    assert "invite_code" in data

    group = Group.objects.get(id=data["id"])
    member = GroupMember.objects.get(group=group,
user=test_user_groups)
    assert member.rank == "ADMIN"
```

Várt Eredmény	HTTP 200-as státuszkód. A válasz tartalmazza a generált meghívó kódot, az adatbázisban pedig létrejön a kapcsolat, ahol a létrehozó ADMIN rangot kap.
Eredmény	Sikeres

Csoporthoz csatlakozás

Végpont	`POST /api/groups/join`
Leírás	Többfelhasználós folyamat tesztelése: "A" felhasználó létrehoz egy csoportot, "B" felhasználó pedig csatlakozik hozzá a generált kód alapján.
Előfeltétel	Létezik egy "Kémia 8/A" nevű csoport. A 2. felhasználó be van jelentkezve, ismeri a kódot, de még nem tagja a csoportnak.
Teszt	

```
def test_join_group(groups_client, auth_headers_groups,
test_user_groups, auth_headers_2_groups,
test_user_2_groups):
    """Test joining a group using a valid invite code."""
    payload = {"name": "Kémia 8/A", "color": "#0000FF"}
    create_response = groups_client.post("/",
json=payload, headers=auth_headers_groups)
    invite_code = create_response.json()["invite_code"]
    group_id = create_response.json()["id"]

    join_payload = {"invite_code": invite_code}
    join_response = groups_client.post("/join",
json=join_payload, headers=auth_headers_2_groups)

    assert join_response.status_code == 200
    assert join_response.json()["name"] == "Kémia 8/A"

    group = Group.objects.get(id=group_id)
    member = GroupMember.objects.get(group=group,
user=test_user_2_groups)
    assert member.rank == "MEMBER"
```

Várt Eredmény	HTTP 200-as státuszkód a csatlakozáskor. A 2. felhasználó bekerül a csoportba, és az adatbázisban automatikusan MEMBER (tag) rangot kap.
Eredmény	Sikeres

Csoport módosítása

Végpont	`PATCH /api/groups/{id}`
Leírás	Egy létező csoport adatainak (név, színkód) részleges frissítése.
Előfeltétel	A csoport létezik, és a kérést küldő felhasználó a csoport adminisztrátora.

Teszt	`def test_update_group_settings(groups_client, auth_headers_groups, test_user_groups):`
	`    """Test updating a group's settings (requires ADMIN rank)."""`
	`    payload = {"name": "Történelem 11/I", "color": "#333333"}`
	`    create_response = groups_client.post("/", json=payload, headers=auth_headers_groups)`
	`    group_id = create_response.json()["id"]`
	
	`    update_payload = {"name": "Történelem 12/I", "color": "#444444"}`
	`    update_response = groups_client.patch(f"/{group_id}", json=update_payload, headers=auth_headers_groups)`
	
	`    assert update_response.status_code == 200`
	`    data = update_response.json()`
	`    assert data["name"] == "Történelem 12/I"`
	`    assert data["color"] == "#444444"`

Várt Eredmény	HTTP 200-as státuszkód. A válaszban az új, frissített név ("Történelem 12/I") és az új színkód ("#444444") szerepel, a módosítások pedig elmentődnek.
Eredmény	Sikeres

2.3. Projektek

Projekt létrehozása

Végpont	`POST /api/projects`
Leírás	Új, egyelőre üres projekt létrehozása. A teszt ellenőrzi, hogy a rendszer automatikusan a kérést küldő felhasználóhoz rendeli-e (creator) a rekordot.
Előfeltétel	A felhasználó be van jelentkezve (érvényes JWT token).
Teszt	
Várt Eredmény	HTTP 200-as státuszkód. Az adatbázisban létrejön a projekt a megadott névvel és leírással, az alkotó (creator) pedig az aktuális tesztfelhasználó lesz.
Eredmény	Sikeres

```python
def test_create_project(blueprints_client,
auth_headers_blueprints, test_user_blueprints):
    """Test successful project creation."""
    payload = {
        "name": "Test Project",
        "desc": "This is a test project"
    }

    response = blueprints_client.post("/", json=payload,
headers=auth_headers_blueprints)

    assert response.status_code == 200
    data = response.json()
    assert data["name"] == "Teszt Projekt"

    project = Project.objects.get(id=data["id"])
    assert project.name == "Teszt Projekt"
    assert project.creator == test_user_blueprints
```

Projekt részleteinek lekérdezése

Végpont	`GET /api/projects/{id}`
Leírás	Egy adott projekt adatainak és a hozzá tartozó beágyazott kérdéseknek/válaszoknak a lekérése azonosító (ID) alapján.
Előfeltétel	Létezik az adatbázisban a projekt, és azt a lekérdezést indító bejelentkezett felhasználó hozta létre.

Teszt	`def test_get_project_details(blueprints_client, auth_headers_blueprints, test_user_blueprints):`
	`    """Test retrieving full project details."""`
	`    project = Project.objects.create(name="Projekt", creator=test_user_blueprints)`
	
	`    response = blueprints_client.get(f"/{project.id}/", headers=auth_headers_blueprints)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["id"] == project.id`
	`    assert data["name"] == "Projekt"`

Várt Eredmény	HTTP 200-as státuszkód. A JSON válaszban a megfelelő projekt adatai (ID, név) térnek vissza.
Eredmény	Sikeres

Projekt frissítése

Végpont	`PUT /api/projects/{id}`
Leírás	Beágyazott (nested) adatszerkezetek frissítése: A teszt ellenőrzi, hogy egy kérésen belül képes-e a rendszer frissíteni a projekt nevét, létrehozni egy új kérdésblokkot, generálni hozzá válaszokat, valamint lekezeli-e a frontend ("single") és a backend ("single_choice") típusok közötti konverziót.
Előfeltétel	A projekt létezik. A payload egy összetett struktúrát tartalmaz (projekt név + 1 kérdés + 2 válasz).
Teszt	

```
def
test_update_full_project_with_blocks(blueprints_client,
auth_headers_blueprints, test_user_blueprints):
    """Test complex update logic: replacing project name
and managing blocks/answers."""
    project = Project.objects.create(name="Régi Név",
creator=test_user_blueprints)

    payload = {
        "name": "Új Név",
        "desc": "Új Leírás",
        "blocks": [
            {
                "type": "single",
                "maintext": "Mennyi 2+2?",
                "subtext": "",
                "answers": [
                    {"text": "3", "is_correct": False,
"points": 0},
                    {"text": "4", "is_correct": True,
"points": 1}
                ]
            }
        ]
    }

    response = blueprints_client.put(f"/{project.id}/",
json=payload, headers=auth_headers_blueprints)
```

	``` assert response.status_code == 200 data = response.json() assert data["name"] == "Új Név" project.refresh_from_db() assert project.name == "Új Név" assert project.blocks.count() == 1 block = project.blocks.first() assert block.type == "single_choice" assert block.maintext == "What is 2+2?" assert block.answers.count() == 2 ```
Várt Eredmény	HTTP 200-as státuszkód. Az adatbázis ellenőrzésekor a projekt neve sikeresen megváltozott, létrejött egy `single_choice` típusú kérdésblokk, amihez pontosan 2 válasz rekord tartozik.
Eredmény	Sikeres

2.4. Kvízek

Új kvíz létrehozása

Végpont	`POST /api/quizzes/`
Leírás	Tanári/Admin funkció tesztelése: egy meglévő tesztsablon (Project) hozzárendelése egy csoporthoz (Group) egy konkrét időablakkal (kezdés és befejezés).
Előfeltétel	A kérést küldő felhasználó a csoport ADMIN-ja (tanár). A kezdési és befejezési dátumok logikailag helyesek (a befejezés a kezdés után van).

Teszt	`def test_create_quiz(quizzes_client, teacher_auth_headers, test_group, test_project):`
	`  """Test successful quiz creation by a teacher."""`
	`  start_time = timezone.now() + timedelta(days=1)`
	`  end_time = start_time + timedelta(hours=1)`
	
	`  payload = {`
	`  "group_id": test_group.id,`
	`  "project_id": test_project.id,`
	`  "date_start": start_time.isoformat(),`
	`  "date_end": end_time.isoformat()`
	`  }`
	
	`  response = quizzes_client.post("/", json=payload, headers=teacher_auth_headers)`
	
	`  assert response.status_code == 200`
	`  data = response.json()`
	`  assert data["project"]["name"] == "Teszt Projekt"`
	
	`  quiz = Quiz.objects.get(id=data["id"])`
	`  assert quiz.group == test_group`
	`  assert quiz.project == test_project`

Várt Eredmény	HTTP 200-as státuszkód. Az adatbázisban a Quiz táblában létrejön a rekord, ami sikeresen összeköti a projektet, a csoportot és az időpontokat.
Eredmény	Sikeres

Kvíz lekérése és indítása (diáknak)

Végpont	`GET /api/quizzes/{id}/start`
Leírás	Annak tesztelése, hogy a diák megkapja-e a vizsga kérdéseit, és hogy az adatok biztonságosan (a helyes megoldásokat elrejtve) érkeznek-e meg a frontend felé.
Előfeltétel	A vizsga státusza aktív (a jelenlegi időpont a start és end dátum közé esik), a kérést küldő diák pedig tagja a csoportnak, de még nem küldte be a vizsgát.

Teszt	`def test_start_quiz(quizzes_client, student_auth_headers, test_group, test_project):`
	`    """Test student getting quiz questions (start quiz)."""`
	`    now = timezone.now()`
	`    quiz = Quiz.objects.create(`
	`        group=test_group,`
	`        project=test_project,`
	`        date_start=now - timedelta(minutes=5),`
	`        date_end=now + timedelta(hours=1)`
	`    )`
	
	`    response = quizzes_client.get(f"/{quiz.id}/start", headers=student_auth_headers)`
	
	`    assert response.status_code == 200`
	`    data = response.json()`
	`    assert data["project"]["name"] == "Teszt Projekt"`
	`    assert len(data["project"]["blocks"]) == 1`
	
	`    assert data["project"]["blocks"][0]["maintext"] == "1+1?"`

Várt Eredmény	HTTP 200-as státuszkód. A válasz JSON-ben szerepelnek a kérdések és a válaszlehetőségek, de a szenzitív adatok (pl. `is_correct` mező) a sémának köszönhetően nem kerülnek kiküldésre.
Eredmény	Sikeres

Kvíz beadása és automatikus értékelése

Végpont	`GET /api/quizzes/{id}/start`
Leírás	Annak tesztelése, hogy a diák megkapja-e a vizsga kérdéseit, és hogy az adatok biztonságosan (a helyes megoldásokat elrejtve) érkeznek-e meg a frontend felé.
Előfeltétel	A vizsga státusza aktív (a jelenlegi időpont a start és end dátum közé esik), a kérést küldő diák pedig tagja a csoportnak, de még nem küldte be a vizsgát.
Teszt	

```
def test_submit_quiz(quizzes_client,
student_auth_headers, test_group, test_project):
    """Test a student submitting a quiz successfully."""
    now = timezone.now()
    quiz = Quiz.objects.create(
        group=test_group,
        project=test_project,
        date_start=now - timedelta(minutes=5),
        date_end=now + timedelta(hours=1)
    )

    block = test_project.blocks.first()
    correct_answer =
block.answers.filter(is_correct=True).first()

    payload = {
        "quiz_id": quiz.id,
        "answers": [
            {
                "block_id": block.id,
                "option_id": correct_answer.id,
                "answer_text": None
            }
        ]
    }

    response = quizzes_client.post("/submit/",
json=payload, headers=student_auth_headers)
```

	``` assert response.status_code == 200 data = response.json() assert data["percentage"] == "100.00" submission = Submission.objects.get(id=data["id"]) assert submission.score == 1.0 assert float(submission.percentage) == 100.0 ```
Várt Eredmény	HTTP 200-as státuszkód. A válasz JSON-ben szerepelnek a kérdések és a válaszlehetőségek, de a szenzitív adatok (pl. `is_correct` mező) a sémának köszönhetően nem kerülnek kiküldésre.
Eredmény	Sikeres